

ASSIGNMENT 01

Date: 10/04/26

Problem: Write a menu driven program that inputs 2 vectors u and v of dimension 3 and perform the following operations: Vector addition, Scalar multiplication, Dot product, Cross product, Orthogonality, Parallelism check.

Source code:

```
import numpy as np

# Input Vector u
print("Enter 3 values for Vector u")
u = np.array([
    float(input("u1 = ")),
    float(input("u2 = ")),
    float(input("u3 = "))
])

print("\nEnter 3 values for Vector v")
v = np.array([
    float(input("v1 = ")),
    float(input("v2 = ")),
    float(input("v3 = "))
])

while True:
    print("\n===== MENU =====")
    print("1. Vector Addition")
    print("2. Scalar Multiplication")
    print("3. Dot Product")
    print("4. Cross Product")
    print("5. Orthogonality Check")
    print("6. Parallelism Check")
    print("7. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        print("\nVector Addition =", u + v)

    elif choice == 2:
        s = float(input("Enter scalar value: "))
        print("\nScalar Multiplication of u =", s * u)
        print("Scalar Multiplication of v =", s * v)

    elif choice == 3:
        print("\nDot Product =", np.dot(u, v))

    elif choice == 4:
        print("\nCross Product =", np.cross(u, v))

    elif choice == 5:
```

```

        if np.isclose(np.dot(u, v), 0):
            print("\nVectors are Orthogonal")
        else:
            print("\nVectors are Not Orthogonal")

    elif choice == 6:
        if np.isclose(np.linalg.norm(np.cross(u, v)), 0):
            print("\nVectors are Parallel")
        else:
            print("\nVectors are Not Parallel")

    elif choice == 7:
        print("\nProgram Ended")
        break

    else:
        print("\nInvalid Choice! Try Again.")

```

Output:

```

Enter 3 values for Vector u u1 = 2 u2 = 3 u3 = 4
Enter 3 values for Vector v
v1 = 1 v2 = 5 v3 = 2
===== MENU =====
Vector Addition
Scalar Multiplication
Dot Product
Cross Product
Orthogonality Check
Parallelism Check 7. Exit
Enter your choice: 1
Vector Addition = [3. 8. 6.]
Enter your choice: 2
Enter scalar value: 3
Scalar Multiplication of u = [ 6.  9. 12.]
Scalar Multiplication of v = [ 3. 15.  6.]
Enter your choice: 3 Dot Product = 25.0
Enter your choice: 4
Cross Product = [-14.  0.  7.]
Enter your choice: 5
Vectors are Not Orthogonal
Enter your choice: 6
Vectors are Not Parallel
Enter your choice: 7
Program Ended

```

ASSIGNMENT 02

Date: 22/04/26

Problem: Write a program that inputs a square matrix of order N and find determinant, inverse, transpose, scalar multiplication, adjoint, rank, diagonal elements, trace eigen value and vector of the input matrix.

Source Code:

```
import numpy as np

n = int(input("Enter the order of square matrix: "))
print("Enter the elements of matrix row wise:")

A = []
for i in range(n):
    row = []
    for j in range(n):
        value = float(input(f"A[{i}][{j}] = "))
        row.append(value)
    A.append(row)
A = np.array(A)
while True:
    print("\n===== MENU =====")
    print("1. Determinant")
    print("2. Inverse")
    print("3. Transpose")
    print("4. Scalar Multiplication")
    print("5. Adjoint")
    print("6. Rank")
    print("7. Diagonal Elements")
    print("8. Trace")
    print("9. Eigen Values and Eigen Vectors")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        print("\nDeterminant =", np.linalg.det(A))

    elif choice == 2:
        det = np.linalg.det(A)
        if np.isclose(det, 0):
            print("\nInverse does not exist.")
        else:
            print("\nInverse Matrix =")
            print(np.linalg.inv(A))

    elif choice == 3:
        print("\nTranspose Matrix =")
        print(A.T)

    elif choice == 4:
        s = float(input("Enter scalar value: "))
```

```

        print("\nResult =")
        print(s * A)

    elif choice == 5:
        det = np.linalg.det(A)
        if np.isclose(det, 0):
            print("\nAdjoint cannot be found.")
        else:
            adj = det * np.linalg.inv(A)
            print("\nAdjoint Matrix =")
            print(np.round(adj))

    elif choice == 6:
        print("\nRank =", np.linalg.matrix_rank(A))

    elif choice == 7:
        print("\nDiagonal Elements =", np.diag(A))

    elif choice == 8:
        print("\nTrace =", np.trace(A))

    elif choice == 9:
        values, vectors = np.linalg.eig(A)
        print("\nEigen Values =")
        print(values)
        print("\nEigen Vectors =")
        print(vectors)

    elif choice == 10:
        print("\nProgram Ended")
        break

    else:
        print("\nInvalid Choice!")

```

Output:

```

Enter the order of square matrix: 2
Enter the elements of matrix row wise:
A[0][0] = 4
A[0][1] = 2
A[1][0] = 1
A[1][1] = 3
===== MENU =====
Determinant
Inverse
Transpose
Scalar Multiplication
Adjoint
Rank
Diagonal Elements
Trace
Eigen Values and Eigen Vectors
Exit
Enter your choice: 1
Determinant = 10.000000000000002

```

ASSIGNMENT 03

Date: 05/05/26

Problem: Write a menu driven program that inputs two matrices A and B and perform the following operations: Addition, subtraction, multiplication, division.

Source Code:

```
import numpy as np

# Input number of rows
r = int(input("Enter number of rows: "))

# Input number of columns
c = int(input("Enter number of columns: "))

# ----- Matrix A -----
print("\nEnter elements of Matrix A")

A = []
for i in range(r):
    row = []
    for j in range(c):
        value = float(input(f"A[{i}][{j}] = "))
        row.append(value)
    A.append(row)

A = np.array(A)

# ----- Matrix B -----
print("\nEnter elements of Matrix B")

B = []
for i in range(r):
    row = []
    for j in range(c):
        value = float(input(f"B[{i}][{j}] = "))
        row.append(value)
    B.append(row)

B = np.array(B)

# Menu Loop
while True:
    print("\n===== MENU =====")
    print("1. Addition")
    print("2. Subtraction")
    print("3. Multiplication")
    print("4. Division")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
```

```

        print("\nAddition =")
        print(np.add(A, B))

    elif choice == 2:
        print("\nSubtraction =")
        print(np.subtract(A, B))

    elif choice == 3:
        if A.shape[1] == B.shape[0]:
            print("\nMultiplication =")
            print(np.matmul(A, B))
        else:
            print("\nMatrix multiplication not possible.")

    elif choice == 4:
        if r == c:
            det = np.linalg.det(B)
            if np.isclose(det, 0):
                print("\nDivision not possible.")
            else:
                invB = np.linalg.inv(B)
                print("\nDivision =")
                print(np.matmul(A, invB))
        else:
            print("\nDivision possible only for square matrices.")

    elif choice == 5:
        print("\nProgram Ended")
        break

    else:
        print("\nInvalid Choice!")

```

Output:

Enter number of rows: 2	Enter your choice: 1
Enter number of columns: 2	Addition =
Enter elements of Matrix A	[[6. 3.]
A[0][0] = 4	[2. 5.]]
A[0][1] = 2	Enter your choice: 2
A[1][0] = 1	Subtraction =
A[1][1] = 3	[[2. 1.]
	[0. 1.]]
Enter elements of Matrix B	Enter your choice: 3
	Multiplication =
B[0][0] = 2	[[10. 8.]
B[0][1] = 1	[5. 7.]]
B[1][0] = 1	Enter your choice: 4
B[1][1] = 2	Division =
===== MENU =====	[[2.33333333 -0.16666667]
Addition	[-0.33333333 1.66666667]]
Subtraction	Enter your choice: 5
Multiplication	Program Ended
Division	
Exit	

ASSIGNMENT 04

Date: 08/05/26

Problem: Consider the vectors $v_1=(1,1,0)$, $v_2=(0,1,1)$ and $v_3=(1,2,1)$ in R_3 . Check whether they are linearly independent. Find a basis for the subspace they span and determine its dimension.

Source Code:

```
from sympy import Matrix

v1 = [1, 1, 0]
v2 = [0, 1, 1]
v3 = [1, 2, 1]

print("v1 =", v1)
print("v2 =", v2)
print("v3 =", v3)

A = Matrix([
    [1, 0, 1],
    [1, 1, 2],
    [0, 1, 1]
])

print("\nMatrix A =")
print(A)

r = A.rank()
print("\nRank of Matrix =", r)

if r == 3:
    print("\nVectors are Linearly Independent")
else:
    print("\nVectors are Linearly Dependent")

rref_matrix, pivot_columns = A.rref()

print("\nRREF of Matrix =")
print(rref_matrix)

print("\nPivot Columns =", pivot_columns)

basis = []
for i in pivot_columns:
    basis.append(A.col(i))

print("\nBasis for the Subspace =")
for b in basis:
    print(b)

dimension = len(basis)
print("\nDimension of the Subspace =", dimension)
```

Output:

```
v1 = [1, 1, 0] v2 = [0, 1, 1] v3 = [1, 2, 1]
Matrix A = Matrix([
[1, 0, 1],
[1, 1, 2],
[0, 1, 1]
])
Rank of Matrix = 2
Vectors are Linearly Dependent
RREF of Matrix =
Matrix([
[1, 0, 1],
[0, 1, 1],
[0, 0, 0]
])
Pivot Columns = (0, 1)
Basis for the Subspace =
Matrix([
[1],
[1],
[0]])
Matrix([
[0],
[1],
[1]])
Dimension of the Subspace = 2
```


ASSIGNMENT 05

Date:12/5/26

Problem: Use Gauss Elimination to solve the following equations and verify the solution with NumPy's built in solver. $4x_1+3x_2-5x_3=2$; $-2x_1-4x_2+5x_3=5$; $8x_1+8x_2=-3$

Source Code:

```
import numpy as np

# Create augmented matrix
A = np.array([
    [4, 3, -5, 2],
    [-2, -4, 5, 5],
    [8, 8, 0, -3]
], dtype=float)

print("Augmented Matrix =\n")
print(A)

# Step 1
A[0] = A[0] / A[0][0]
print("\nAfter R1 = R1 / 4\n")
print(A)

# Step 2
A[1] = A[1] + 2 * A[0]
A[2] = A[2] - 8 * A[0]

print("\nAfter making first column zero\n")
print(A)

# Step 3
A[1] = A[1] / A[1][1]

print("\nAfter making second pivot 1\n")
print(A)

# Step 4
A[2] = A[2] - A[2][1] * A[1]

print("\nAfter making second column zero\n")
print(A)

# Step 5
A[2] = A[2] / A[2][2]

print("\nAfter making third pivot 1\n")
print(A)

# Back substitution
x3 = A[2][3]
x2 = A[1][3] - A[1][2] * x3
x1 = A[0][3] - A[0][1] * x2 - A[0][2] * x3
```

```

print("\nSolution using Gauss Elimination")
print("x1 =", x1)
print("x2 =", x2)
print("x3 =", x3)

# Verification using NumPy
C = np.array([
    [4, 3, -5],
    [-2, -4, 5],
    [8, 8, 0]
], dtype=float)

B = np.array([2, 5, -3], dtype=float)

solution = np.linalg.solve(C, B)

print("\nSolution using np.linalg.solve()")
print(solution)

```

Output:

```

Augmented Matrix =
[[ 4.  3. -5.  2.]
 [-2. -4.  5.  5.]
 [ 8.  8.  0. -3.]]
After R1 = R1 / 4
[[ 1.  0.75 -1.25  0.5 ]
 [-2. -4.  5.  5. ]
 [ 8.  8.  0. -3. ]]
After making first column zero
[[ 1.  0.75 -1.25  0.5 ]
 [ 0. -2.5  2.5  6. ]
 [ 0.  2.  10. -7. ]]
After making second pivot 1
[[ 1.  0.75 -1.25  0.5 ]
 [-0.  1. -1. -2.4 ]
 [ 0.  2.  10. -7. ]]
After making second column zero
[[ 1.  0.75 -1.25  0.5 ]
 [-0.  1. -1. -2.4 ]
 [ 0.  0.  12. -2.2 ]]
After making third pivot 1
[[ 1.  0.75 -1.25  0.5 ]
 [-0.  1. -1. -2.4 ]
 [ 0.  0.  1. -0.18333333]]
Solution using Gauss Elimination x1 = 2.0708333333333333
x2 = -2.5833333333333335 x3 = -0.18333333333333335
Solution using np.linalg.solve()
[ 2.07083333 -2.58333333 -0.18333333]

```

ASSIGNMENT 06

Date:30/5/26

Problem: Solve the following system of linear equations using Gaussian elimination and Gauss-Jordan method and determine whether the system is consistent or inconsistent: $x+y+z=6$; $2x+3y+z=10$; $x+2y+2z=9$

Source Code:

```
from sympy import Matrix

# Create augmented matrix
A = Matrix([
    [1, 1, 1, 6],
    [2, 3, 1, 10],
    [1, 2, 2, 9]
])

print("Original Augmented Matrix =\n")
print(A)

# ----- Gaussian Elimination -----
A[1, :] = A[1, :] - 2 * A[0, :]
A[2, :] = A[2, :] - A[0, :]

print("\nAfter First Elimination =\n")
print(A)

A[2, :] = A[2, :] - A[1, :]

print("\nAfter Second Elimination =\n")
print(A)

# Back Substitution
z = A[2, 3] / A[2, 2]
y = (A[1, 3] - A[1, 2] * z) / A[1, 1]
x = A[0, 3] - y - z

print("\nSolution using Gaussian Elimination")
print("x =", x)
print("y =", y)
print("z =", z)

# ----- Gauss-Jordan -----
B = Matrix([
    [1, 1, 1, 6],
    [2, 3, 1, 10],
    [1, 2, 2, 9]
])

rref_matrix, pivot = B.rref()

print("\nReduced Row Echelon Form =\n")
print(rref_matrix)
```

```

# Check consistency
consistent = True

for row in rref_matrix.tolist():
    if row[0] == 0 and row[1] == 0 and row[2] == 0 and row[3] != 0:
        consistent = False

if consistent:
    print("\nSystem is Consistent")
else:
    print("\nSystem is Inconsistent")

print("\nSolution using Gauss-Jordan Method")
print("x =", rref_matrix[0, 3])
print("y =", rref_matrix[1, 3])
print("z =", rref_matrix[2, 3])

```

Output:

```

Original Augmented Matrix =
Matrix([
[1, 1, 1, 6],
[2, 3, 1, 10],
[1, 2, 2, 9]])
After First Elimination =
Matrix([
[1, 1, 1, 6],
[0, 1, -1, -2],
[0, 1, 1, 3]])
After Second Elimination =
Matrix([
[1, 1, 1, 6],
[0, 1, -1, -2],
[0, 0, 2, 5]])
Solution using Gaussian Elimination  $x = 1/2$   $y = 1/2$   $z = 5/2$ 

Reduced Row Echelon Form =
Matrix([
[1, 0, 0, 1/2],
[0, 1, 0, 1/2],
[0, 0, 1, 5/2]])
System is Consistent Solution using Gauss-Jordan Method
 $x = 1/2$   $y = 1/2$   $z = 5/2$ 

```

ASSIGNMENT 07

Date: 06/06/26

Problem : Consider the following small dataset containing information about wheat seeds:

Area	Perimeter	Compactness	Kernel Length	Kernel Width
15.26	14.84	0.8710	5.763	3.312
14.88	14.57	0.8811	5.554	3.333
14.29	14.09	0.9050	5.291	3.337
13.84	13.94	0.8955	5.324	3.379
16.14	14.99	0.9034	5.658	3.562

Identify the data type of each variable and compute appropriate measures of central tendency and dispersion for the numerical variables. Use Pandas functions like `df.describe()`, `mean()`, `median()`, and `std()` to compute the summary statistics.

Source Code:

```
import pandas as pd

data = {
    "area": [15.26, 14.88, 14.29, 13.84, 16.14],
    "perimeter": [14.84, 14.57, 14.09, 13.94, 14.99],
    "compactness": [0.8710, 0.8811, 0.9050, 0.8955, 0.9034],
    "kernel_length": [5.763, 5.554, 5.291, 5.324, 5.658],
    "kernel_width": [3.312, 3.333, 3.337, 3.379, 3.562]
}

df = pd.DataFrame(data)

print("Dataset:")
print(df)

print("\nData Types:")
print(df.dtypes)

# Summary Statistics
print("\nSummary Statistics:")
print(df.describe())

# Mean
print("\nMean:")
print(df.mean())

# Median
print("\nMedian:")
print(df.median())

print("\nStandard Deviation:")
print(df.std())
```

Output:

Dataset:

	area	perimeter	compactness	kernel_length	kernel_width
0	15.26	14.84	0.8710	5.763	3.312
1	14.88	14.57	0.8811	5.554	3.333
2	14.29	14.09	0.9050	5.291	3.337
3	13.84	13.94	0.8955	5.324	3.379
4	16.14	14.99	0.9034	5.658	3.562

Data Types:

area	float64
perimeter	float64
compactness	float64
kernel_length	float64
kernel_width	float64
dtype:	object

Summary Statistics:

	area	perimeter	compactness	kernel_length	kernel_width
count	5.000000	5.000000	5.000000	5.000000	5.000000
mean	14.882000	14.486000	0.891200	5.518000	3.384600
std	0.853194	0.448475	0.015034	0.205791	0.101056
min	13.840000	13.940000	0.871000	5.291000	3.312000
25%	14.290000	14.090000	0.881100	5.324000	3.333000
50%	14.880000	14.570000	0.895500	5.554000	3.337000
75%	15.260000	14.840000	0.903400	5.658000	3.379000
max	16.140000	14.990000	0.905000	5.763000	3.562000

Mean:

area	14.8820
perimeter	14.4860
compactness	0.8912
kernel_length	5.5180
kernel_width	3.3846
dtype:	float64

Median:

area	14.8800
perimeter	14.5700
compactness	0.8955
kernel_length	5.5540
kernel_width	3.3370
dtype:	float64

Standard Deviation:

area	0.853194
perimeter	0.448475
compactness	0.015034
kernel_length	0.205791
kernel_width	0.101056
dtype:	float64

ASSIGNMENT 08

Date: 06/06/26

Problem: Draw suitable diagrams to represent both absolute and cumulative frequencies for the following distribution of family size for 100 families.

Family Size	2	3	4	5	6	7	8	9	10
Frequency	26	21	17	12	9	3	2	2	1

Source Code:

```
import matplotlib.pyplot as plt
import numpy as np

family_size = [2, 3, 4, 5, 6, 7, 8, 9, 10]
frequencies = [26, 21, 17, 12, 9, 3, 2, 2, 1]

# Absolute Frequency Diagram
plt.figure(figsize=(8, 5))

plt.bar(
    family_size,
    frequencies,
    color="skyblue",
    edgecolor="black"
)

plt.title("Absolute Frequency Distribution")
plt.xlabel("Family Size")
plt.ylabel("Frequency")
plt.grid(axis="y", linestyle="--", alpha=0.5)

plt.show()

# Cumulative Frequency Diagram
cumulative_frequency = np.cumsum(frequencies)

plt.figure(figsize=(8, 5))

plt.plot(
    family_size,
    cumulative_frequency,
    marker="o",
    linewidth=2,
    color="red"
)

plt.title("Cumulative Frequency Distribution")
plt.xlabel("Family Size")
plt.ylabel("Cumulative Frequency")
plt.grid(True)

plt.show()
```

Output:

